

Concevez et analyser une base de données

NoSQL

Hyani Hishem

Contexte

L'association **NosCités**

- Surveille les plateformes de locations de courte durée dans plusieurs villes en France (Paris et Lyon)
- Évaluer l'impact de ces plateformes sur l'offre de logements.

Problématique :

- Crash total de la base de données des locations à Paris suite à une insuffisance de précautions.

Objectifs de la mission :

- **Restaurer la base de données** MongoDB
- **Analyser son intégrité** par toute une série de requêtes
- **Pérenniser son utilisation** pour prévenir de futurs incidents

Partie 1: Base de données NoSQL

Les avantages du NoSQL dans ce contexte de données concernent:

1) Flexibilité :

- **Pas de schéma strict** : MongoDB est schema-less

2) Haute disponibilité :

- **Replica Set** : plusieurs copies de la base pour garantir l'accès même en cas de panne.

3) Scalabilité horizontale :

- **Sharding** : répartition des données sur plusieurs serveurs pour gérer de gros volumes et améliorer les performances.

Partie 1: Base de données NoSQL

Importation:

Delete all 95885 documents retrieved

12345>>

_id	id	listing_url	scrape_id	last_scraped	source	name	description	neighborhood_c
  693c91388d7073d495a7f0e2	80260	https://www.airbnb.com/rooms/80260	20240610195007	Thu Jun 13 2024 00:00:00 GMT+0000 (Coordinated Universal Time)	previous scrape	Nice studio in Jourdain's village		
  693c91388d7073d495a7f0e3	3109	https://www.airbnb.com/rooms/3109	20240610195007	Wed Jun 12 2024 00:00:00 GMT+0000 (Coordinated Universal Time)	city scrape	zen and calm	Lovely Appartment with one bedroom with a Queen s...	Good restaurants />very close the Montparnasse...
  693c91388d7073d495a7f0e4	80301	https://www.airbnb.com/rooms/80301	20240610195007	Thu Jun 13 2024 00:00:00 GMT+0000 (Coordinated Universal Time)	city scrape	toits de Paris	On the top,sharing my space,you have your bedroom...	SAFE neighborhood bus and subway,velib(bike)

Partie 1: Base de données NoSQL

Exemple de document:

```
1 {
2   _id: ObjectId('693c91388d7073d495a7f0e2'),
3   id: 80260,
4   listing_url: 'https://www.airbnb.com/rooms/80260',
5   scrape_id: 20240610195007,
6   last_scraped: ISODate('2024-06-13T00:00:00.000Z'),
7   source: 'previous scrape',
8   name: 'Nice studio in Jourdain\'s village',
9   description: null,
10  neighborhood_overview: null,
11  picture_url: 'https://a0.muscache.com/pictures/716553/6c14f251_original.jpg',
12  host_id: 333548,
13  host_url: 'https://www.airbnb.com/users/show/333548',
14  host_name: 'Charlotte',
15  host_since: ISODate('2011-01-03T00:00:00.000Z'),
16  host_location: 'Paris, France',
17  host_about: 'My name is Charlotte, I\'m 26 and I study cinema production. I\'ll be gone the whole year to study in Grenoble',
18  host_response_time: null,
19  host_response_rate: null,
20  host_acceptance_rate: null,
21  host_is_superhost: false,
22  host_thumbnail_url: 'https://a0.muscache.com/im/users/333548/profile_pic/1350320876/original.jpg?aki_policy=profile_small',
23  host_picture_url: 'https://a0.muscache.com/im/users/333548/profile_pic/1350320876/original.jpg?aki_policy=profile_x_medium',
24  host_neighbourhood: 'Buttes-Chaumont - Belleville',
25  host_listings_count: 1,
26  host_total_listings_count: 1,
27  host_verifications: [
28    'email',
29    'phone'
30  ],
31  host_has_profile_pic: true,
32  host_identity_verified: true,
33  neighbourhood: null,
34  neighbourhood_cleansed: 'Ménilmontant',
35  latitude: 48.87131,
36  longitude: 2.38848,
37  property_type: 'Entire rental unit',
38  room_type: 'Entire home/apt',
39  accommodates: 3,
40  bathrooms: null,
41  bathrooms_text: '1 bath',
42  bedrooms: 1,
43  beds: null,
```

```
44  amenities: [
45    'Hangers',
46    'Essentials',
47    'Wifi',
48    'Dishes and silverware',
49    'Dedicated workspace',
50    'Hair dryer',
51    'Kitchen',
52    'Shampoo',
53    'Pets allowed',
54    'Heating'
55  ],
56  price: null,
57  minimum_nights: 2,
58  maximum_nights: 730,
59  minimum_minimum_nights: 2,
60  maximum_minimum_nights: 2,
61  minimum_maximum_nights: 730,
62  maximum_maximum_nights: 730,
63  minimum_nights_avg_ntm: 2,
64  maximum_nights_avg_ntm: 730,
65  has_availability: true,
66  availability_30: 0,
67  availability_60: 0,
68  availability_90: 0,
69  availability_365: 0,
70  calendar_last_scraped: ISODate('2024-06-13T00:00:00.000Z'),
71  number_of_reviews: 206,
72  number_of_reviews_ltm: 0,
73  number_of_reviews_l30d: 0,
74  first_review: ISODate('2011-04-18T00:00:00.000Z'),
75  last_review: ISODate('2021-10-03T00:00:00.000Z'),
76  review_scores_rating: 4.63,
77  review_scores_accuracy: 4.61,
78  review_scores_cleanliness: 4.75,
79  review_scores_checkin: 4.85,
80  review_scores_communication: 4.78,
81  review_scores_location: 4.61,
82  review_scores_value: 4.64,
83  license: '7512005348473',
84  instant_bookable: false,
85  calculated_host_listings_count: 1,
86  calculated_host_listings_count_entire_homes: 1,
87  calculated_host_listings_count_private_rooms: 0,
88  calculated_host_listings_count_shared_rooms: 0,
89  reviews_per_month: 1.29
90 }
```

Partie 1: Base de données NoSQL

Nombre de documents de la base de données

```
switched to db locations_Paris  
[direct: mongos] locations_Paris> db.listings.countDocuments()  
95885  
[direct: mongos] locations_Paris> █
```

Nombre de logements avec des disponibilités:

```
95885  
[direct: mongos] locations_Paris> db.listings.countDocuments({"has_availability":true})  
90173  
[direct: mongos] locations_Paris> █
```

Partie 2 : Analyse de la BDD NoSQL

Combien d'annonces y a-t-il par type de location ?

```
[direct: mongos] locations_Paris> db.listings.aggregate([ { $group :{_id:"$property_type", count:{$sum :1} } } , {$sort:{count:-1} }])  
[  
  { _id: 'Entire rental unit', count: 80516 },  
  { _id: 'Private room in rental unit', count: 5980 },  
  { _id: 'Entire condo', count: 2679 },  
  { _id: 'Room in boutique hotel', count: 1256 },  
  { _id: 'Entire loft', count: 1007 },  
  { _id: 'Room in hotel', count: 1003 },  
  { _id: 'Entire home', count: 696 },  
  { _id: 'Private room in bed and breakfast', count: 590 },  
  { _id: 'Private room in condo', count: 344 },  
  { _id: 'Room in hotel', count: 344 }  
]
```

Quelles sont les 5 annonces de location avec le plus d'évaluations ? Et combien d'évaluations ont-elles ?

```
[direct: mongos] locations_Paris> db.listings.find({}, {_id:0, id:1, number_of_reviews:1}).sort({ number_of_reviews: -1 }).limit(5)  
[  
  { id: 17222007, number_of_reviews: 3067 },  
  { id: 26244787, number_of_reviews: 2620 },  
  { id: 41020735, number_of_reviews: 2294 },  
  { id: 40194697, number_of_reviews: 2105 },  
  { id: 35145338, number_of_reviews: 2048 }  
]
```

Partie 2 : Analyse de la BDD NoSQL

Quel est le nombre total d'hôtes différents ?

```
[direct: mongos] locations_Paris> db.listings.distinct('host_id').length  
71979
```

Quel est le nombre de locations réservables instantanément ? Cela représente quelle proportion des annonces ?

```
[direct: mongos] locations_Paris> N_document=db.listings.countDocuments()  
95885  
[direct: mongos] locations_Paris> N_document_bookable=db.listings.countDocuments({"instant_bookable":true})  
22094  
[direct: mongos] locations_Paris> proportion = N_document_bookable/N_document  
0.23042185951921573  
[direct: mongos] locations_Paris>
```


Partie 2 : Analyse de la BDD NoSQL

Est-ce que des hôtes ont plus de 100 annonces sur les plateformes ? Et si oui qui sont-ils ? Cela représente quel pourcentage des hôtes ?

```
[direct: mongos] locations_Paris> db.listings.aggregate([ { $group: { _id: '$host_id' , count: { $sum: 1 } } } , { $match: { count: { $gt: 100 } } } , { $sort: { count: -1 } } ] )
```

```
[
  { _id: 314994947, count: 730 },
  { _id: 33889201, count: 497 },
  { _id: 50502817, count: 426 },
  { _id: 50978178, count: 307 },
```

```
[direct: mongos] locations_Paris> sup_host=db.listings.aggregate([ { $group: { _id: '$host_id' , count: { $sum: 1 } } } , { $match: { count: { $gt: 100 } } } , { $sort: { count: -1 } } ] ).toArray().length
```

```
24
```

```
[direct: mongos] locations_Paris> tot_host=db.listings.distinct('host_id').length
```

```
71979
```

```
[direct: mongos] locations_Paris> proportion=sup_host/tot_host
```

```
0.00033343058392031006
```

Partie 2 : Analyse de la BDD NoSQL

Combien y a-t-il de super hôtes différents ? Cela représente quel pourcentage des hôtes ?

```
[direct: mongos] locations_Paris> super_host=db.listings.countDocuments({'host_is_superhost':true})  
14776
```

```
[direct: mongos] locations_Paris> tot_host=db.listings.distinct('host_id').length  
71979
```

```
[direct: mongos] locations_Paris> proportion=super_host/tot_host  
0.20528209616693757
```

Partie 2 : Analyse de la BDD NoSQL (Polars)

Calculer le taux de réservation moyen par mois par type de logement

```
reser_moyen_par_type = (  
    df  
    .group_by("property_type")  
    .agg(((30-pl.col("availability_30"))/30)  
    .mean().alias("taux_reservation_30"))  
    .sort("taux_reservation_30", descending=True)  
)  
  
print(reser_moyen_par_type)
```

property_type	taux_reservation_30
---	---
str	f64
Dome	1.0
Shared room in guest suite	1.0
Shared room in ice dome	1.0
Entire bungalow	1.0
Cave	1.0
...	...
Shipping container	0.033333
Tower	0.033333
Barn	0.0
Entire bed and breakfast	0.0
Castle	0.0

Partie 2 : Analyse de la BDD NoSQL (Polars)

Calculer la médiane des nombre d'avis pour tous les logements

```
median_reviews =(
    df
    .select(pl.col("number_of_reviews")
    .median()
    .alias("median number of reviews")
    )
    )

print(median_reviews)
```

median number of reviews

f64
3.0

Partie 2 : Analyse de la BDD NoSQL (Polars)

Calculer la médiane des nombre d'avis par catégorie d'hôte

```
median_reviewers_superhost =(
    df
    .group_by("host_is_superhost")
    .agg(pl.col("number_of_reviews")
    .median()
    .alias("median number of reviews")
)

print(median_reviewers_superhost)
```

host_is_superhost	median number of reviews
---	---
bool	f64
false	2.0
true	24.0

Partie 2 : Analyse de la BDD NoSQL (Polars)

Calculer la densité de logements par quartier de Paris

```
dens_quartier = (  
    df  
    .group_by("neighbourhood_cleansed")  
    .agg(pl.len().alias("nombre_locations"))  
    .with_columns(  
        (pl.col("nombre_locations")  
         / pl.col("nombre_locations").sum())  
    ).alias("densite")  
    )  
    .sort("densite", descending=True)  
    )  
  
print(dens_quartier)
```

neighbourhood_cleansed	nombre_locations	densite
---	---	---
str	u32	f64
Buttes-Montmartre	10555	0.11008
Popincourt	8430	0.087918
Vaugirard	7802	0.081368
Batignolles-Monceau	6857	0.071513
Entrepôt	6558	0.068394
...	...	...
Élysée	2898	0.030224
Hôtel-de-Ville	2821	0.029421
Palais-Bourbon	2740	0.028576
Luxembourg	2701	0.028169
Louvre	2026	0.021129

Partie 2 : Analyse de la BDD NoSQL (Polars)

Identifier les quartiers avec le plus fort taux de réservation par mois

```
reser_quartier = (  
    df  
    .group_by("neighbourhood_cleansed")  
    .agg(((30-pl.col("availability_30"))/30)  
    .mean()  
    .alias("taux_reservation_30")  
    )  
    .sort("taux_reservation_30", descending=True)  
)  
  
print(reser_quartier)
```

neighbourhood_cleansed	taux_reservation_30
---	---
str	f64
Ménilmontant	0.754152
Entrepôt	0.748135
Popincourt	0.747754
Buttes-Chaumont	0.741269
Panthéon	0.731351
...	...
Palais-Bourbon	0.688686
Bourse	0.686932
Luxembourg	0.664532
Élysée	0.6245
Passy	0.623032

Partie 2 : Analyse de la BDD NoSQL (Metabase)

Service Docker lié à l'outil BI

Configuration de l'outil BI Metabase

```
metabase:  
  image: metabase/metabase:latest  
  container_name: metabase  
  ports:  
    - "3000:3000"  
  depends_on:  
    - mongo  
  environment:  
    MB_DB_FILE: /metabase/metabase.db  
  volumes:  
    - ./metabase-data:/metabase  
  networks:  
    - mongo_net
```

```
volumes:  
  mongo_data:  
  
networks:  
  mongo_net:  
    driver: bridge
```

<http://localhost:3000/>

The screenshot shows the Metabase configuration interface for a new database. The 'Bases de données' (Databases) tab is selected in the top navigation bar. The 'Méthode de connexion' (Connection method) section has two options: 'Remplissez l'hôte, le port et le nom de la base de données' (selected) and 'Utiliser une chaîne de connexion'. The configuration fields are as follows:

- Hôte**: mongo
- Nom de la base de données**: location
- Port**: 27017
- Nom d'utilisateur**: rw
- Mot de passe**: [masked]
- Authentication database (optional)**: admin

Partie 2 : Analyse de la BDD NoSQL (Metabase)

Capture d'écran de l'outil BI connecté à la collection Listings :

projeto07 / Listings

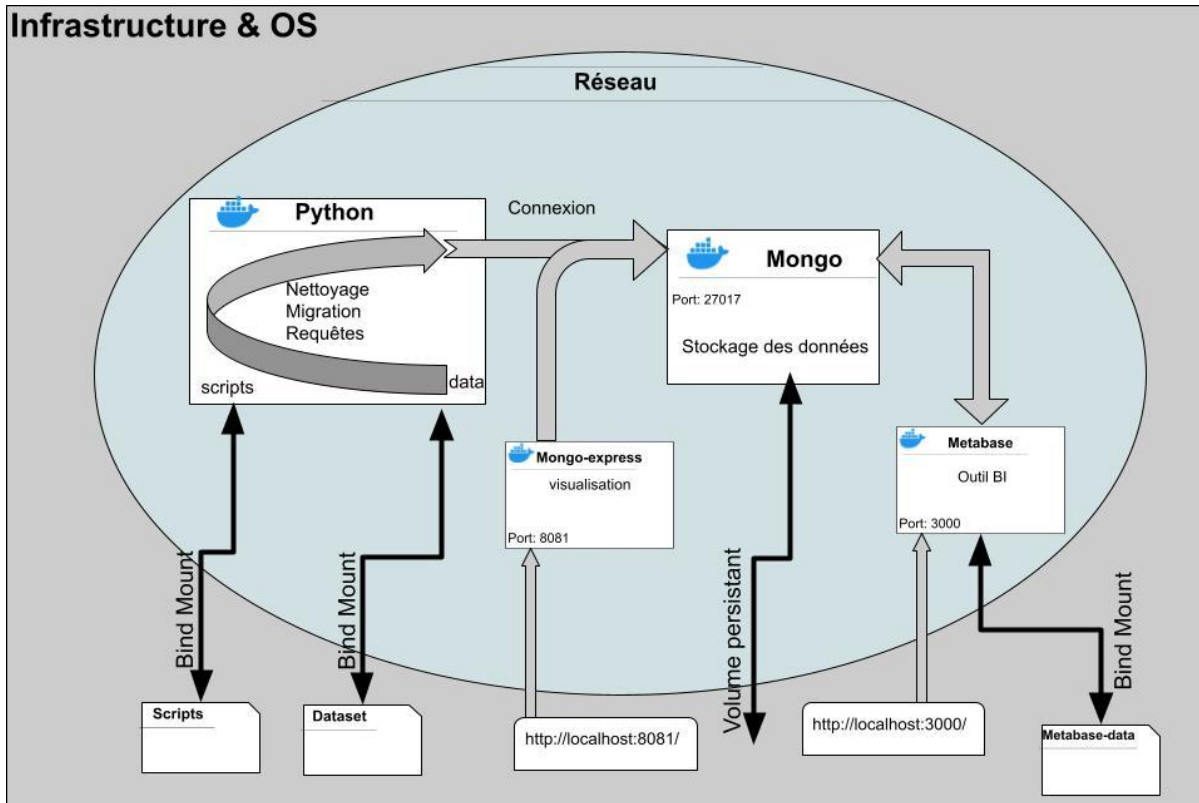
Filtre

Σ Résumé

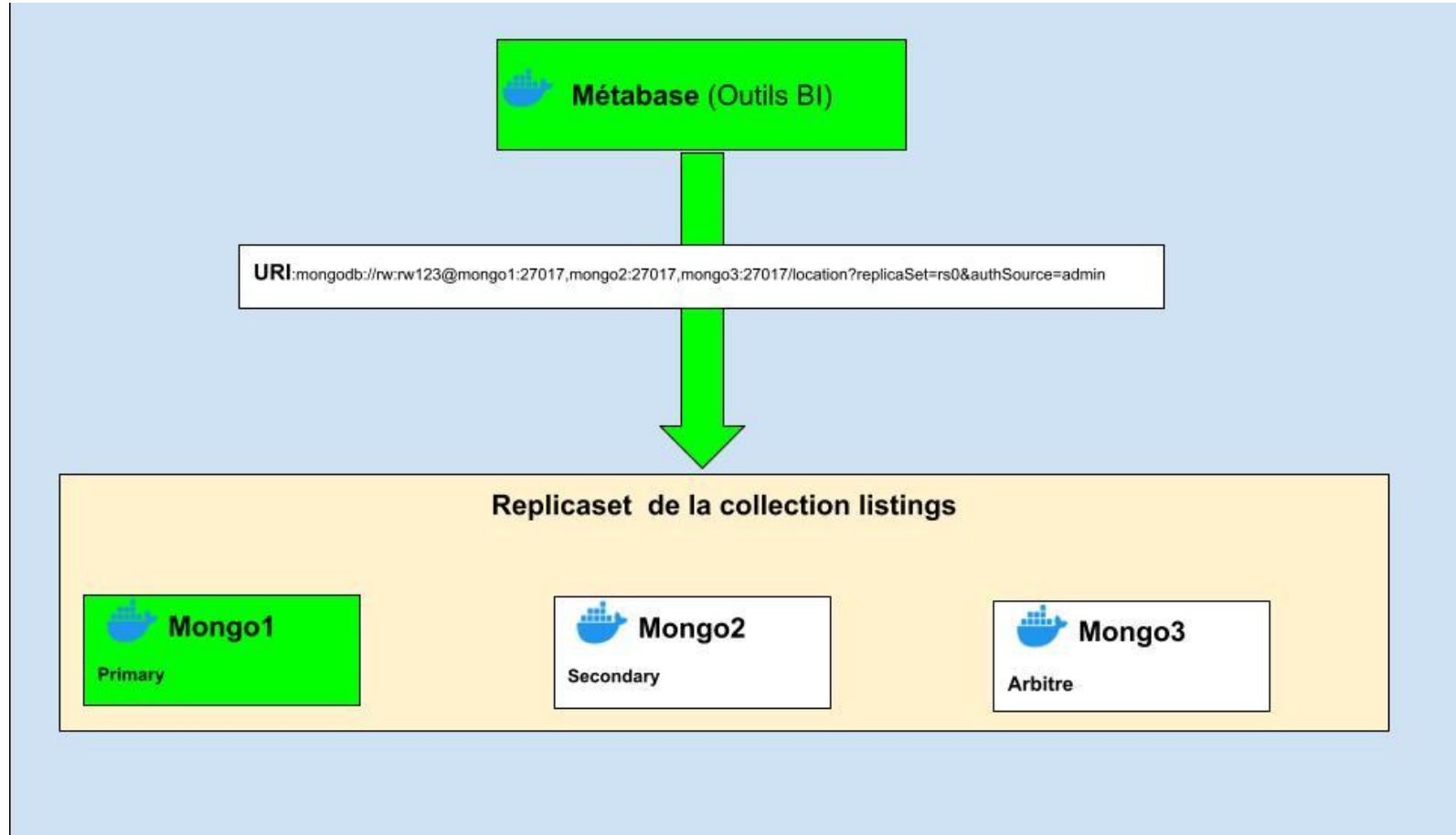
ID	ID	Listing URL	Scrape ID	Last Scraped
693e664e66dadd7d274f16df	80260	https://www.airbnb.com/rooms/80260	20,240,610,195,007	juin 13, 2024, 12:00 AM
693e664e66dadd7d274f16e0	3109	https://www.airbnb.com/rooms/3109	20,240,610,195,007	juin 12, 2024, 12:00 AM
693e664e66dadd7d274f16e1	80301	https://www.airbnb.com/rooms/80301	20,240,610,195,007	juin 13, 2024, 12:00 AM
693e664e66dadd7d274f16e2	5396	https://www.airbnb.com/rooms/5396	20,240,610,195,007	juin 13, 2024, 12:00 AM
693e664e66dadd7d274f16e3	7397	https://www.airbnb.com/rooms/7397	20,240,610,195,007	juin 13, 2024, 12:00 AM
693e664e66dadd7d274f16e4	80327	https://www.airbnb.com/rooms/80327	20,240,610,195,007	juin 12, 2024, 12:00 AM
693e664e66dadd7d274f16e5	7964	https://www.airbnb.com/rooms/7964	20,240,610,195,007	juin 13, 2024, 12:00 AM

Partie 2 : Analyse de la BDD NoSQL (Metabase)

Schéma de l'architecture docker-compose



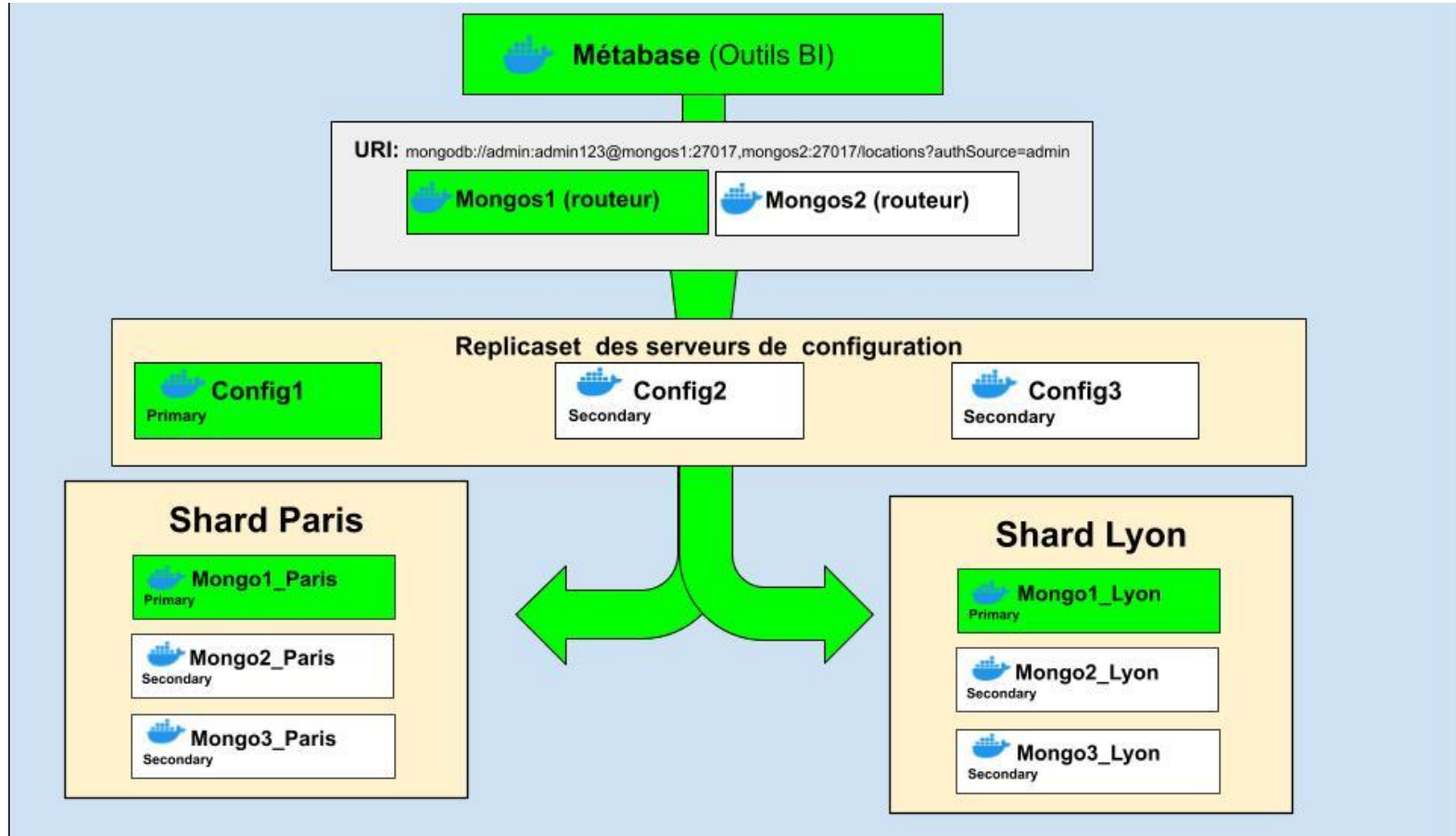
Partie 3 : Replicaset



Partie 3 : Sharding

1. Services docker:
 - 3 conteneurs Config : (Primary +2 Secondary)
 - 3 conteneurs Mongo Paris: (Primary +2 Secondary)
 - 3 conteneurs Mongo Lyon: (Primary +2 Secondary)
 - 2 Conteneurs Mongos routeur pour garantir la tolérance aux pannes
 - Conteneurs applications (Mongo-express, Metabase et python)
2. Entrer dans un conteneur Config puis initialiser le replica set.
3. Entrer dans un conteneur Mongo Paris puis initialiser le replica set.
4. Entrer dans un conteneur Mongo Lyon puis initialiser le replica set.
5. Entrer dans un conteneur Mongos:
 - Ajouter les shards
 - Activer les sharding sur la base de donnée
 - Créer la collection avec le sharding et définir la clé de sharding sur deux champs (city et id_host)
 - Ajouter un tag pour forcer la répartition sur le champs city: lyon dans le shard à Lyon et paris dans le shard à Paris
6. Importer les documents via python

Partie 3 : Sharding



Conclusion

Nous avons :

- restauré la base de donnée suite à l'incident
- analysé l'intégrité des données via requêtes ciblées
- mis en place d'une architecture robuste :
 - a. Haute disponibilité
 - b. Scalabilité